

Interview Evaluation Report

Candidate: piyush
Interview Type: practice
Date: 2026-03-03 12:16

Performance Summary

Category	Score
Overall Score	55/100
Technical Knowledge	20/25
Communication Skills	10/25
Problem Solving	12/25
Project Understanding	0/25

Strengths

- Strong SQL fundamentals: produced a correct, well-structured top-N revenue query and showed awareness of cross-database syntax differences (LIMIT vs TOP, date functions).
- Good Power BI conceptual understanding: clearly distinguished calculated columns vs measures and tied each to appropriate use cases (row-level logic vs filter-responsive aggregations).
- Solid data engineering/analytics methods in SQL: described practical cleaning/validation techniques (deduplication with ROW_NUMBER/CTEs, standardization with string functions, COALESCE for nulls, basic integrity/range checks) and demonstrated advanced segmentation concepts (RFM with NTILE, ABC-XYZ, rolling windows, SCD Type 2).

Areas for Improvement

- Project articulation gaps: struggled to give specific KPIs and an example SQL computation for the marketing report, and could not explain Power BI data modeling/visual drilldowns for the Global Superstore report.
- Behavioral/problem-solving responses were incomplete: discrepancy question lacked the investigation steps, validation approach, and resolution/communication; also showed inability to propose scaling/near-real-time architecture changes.

Suggestions

- Prepare project “story packets” for each portfolio item: 1) business goal, 2) dataset/tables, 3) KPIs with precise formulas, 4) one representative SQL query (with joins and edge cases), 5) results/impact.

Practice delivering this in 60–90 seconds per project.

- For Power BI, rehearse a standard modeling explanation: fact vs dimension tables, star schema, relationship cardinality/filter direction, date table, key DAX measures, and 2–3 visuals with drilldown paths (e.g., Region → Category → Subcategory → Product; top/bottom via ranking and decomposition tree).
- Build a repeatable framework for discrepancy + scaling questions: discrepancy = reconcile definitions, check joins/filters/grain, validate counts against source, segment to isolate issue, document fix; scaling = incremental loads/CDC, partitioning/indexing, materialized aggregates, orchestration (Airflow/dbt), and a semantic layer with pre-aggregations for near-real-time dashboards.

Interview Questions & Answers

Q1: In Python, how do NumPy arrays differ from native Python lists, and when would you prefer NumPy for data analysis?

Answer: NumPy arrays are more efficient for numerical tasks with large datasets due to their speed and compact memory usage, while native Python lists are more flexible for general-purpose programming

Score: 7/10 | **Feedback:** The answer correctly highlights that NumPy arrays are faster and more memory-efficient for large numerical datasets, and that Python lists are more flexible. To be stronger, it should explicitly mention key reasons: NumPy arrays are homogeneous and stored contiguously, and they support vectorized operations via ufuncs and broadcasting, which is why they outperform lists for numerical computation.

Q2: Using Pandas, how would you handle missing values in a DataFrame, and what are the trade-offs between dropping vs imputing?

Answer: In Pandas, missing values (represented as NaN or None) can be handled by either dropping them using the `dropna()` method or imputing them (filling with substitute values) using the `fillna()` or `interpolate()` methods. The choice between dropping and imputing involves trade-offs related to data loss, bias, and data integrity.

Score: 7/10 | **Feedback:** Covers the core Pandas methods (`dropna`, `fillna`, `interpolate`) and correctly notes key trade-offs (data loss, bias, integrity). To be stronger, it should mention how to assess missingness first (`isna()`, `isnull()`, `info()`, missingness patterns) and give concrete imputation strategies (mean/median/mode, forward/back fill, model-based) and when each is appropriate (e.g., time series vs categorical).

Q3: Write or describe a SQL query pattern to find the top 5 products by total revenue in the last 30 days.

Answer: `SELECT product_id, SUM(sale_amount) AS total_revenue -- or SUM(quantity * price) FROM sales WHERE sale_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY) -- Example for MySQL/PostgreSQL -- For SQL Server, use: sale_date >= DATEADD(day, -30, GETDATE()) GROUP BY product_id ORDER BY total_revenue DESC LIMIT 5; -- For MySQL/PostgreSQL -- For SQL Server, place TOP 5 after SELECT: SELECT TOP 5 product_id, ...`

Score: 10/10 | **Feedback:** Covers the full pattern: correct 30-day date filter, SUM aggregation for revenue, GROUP BY product, ORDER BY revenue DESC, and LIMIT/TOP 5. Also appropriately notes dialect differences.

Q4: In Power BI, what is the difference between a calculated column and a measure, and when would you use each?

Answer: In Power BI, calculated columns are computed row-by-row and stored in the model, while measures are calculated on-the-fly based on user filters. Use calculated columns for row-level logic or filtering (slicers), and measures for dynamic aggregations (SUM, AVG) that change with report

interactions

Score: 9/10 | **Feedback:** Accurately distinguishes calculated columns (row-by-row, stored in the model at refresh) from measures (evaluated at query time and responsive to filter context), and gives appropriate use cases. Could be slightly clearer by explicitly naming row context vs. filter context and noting that calculated columns increase model size while measures do not store results.

Q5: For your 'Marketing Analytics Report - Shop Easy (SQL)' project, which KPIs did you track to evaluate campaign performance, and how did you compute one of them in SQL?

Answer: The specific KPIs tracked often depend on the business objectives but usually include: Conversion Rate (CVR): The percentage of visitors who complete a desired action, such as making a purchase. Customer Acquisition Cost (CAC): The total expense incurred to acquire a new customer. Return on Ad Spend (ROAS): The revenue generated for every dollar (or other currency) spent on advertising. Average Order Value (AOV): The average amount spent each time a customer places an order. Customer Lifetime Value (CLV): The total revenue a customer is expected to generate over their entire relationship with the company. Website Traffic & Bounce Rate: To understand the volume and quality of visitors coming from different marketing channels. Cart Abandonment Rate: The percentage of shoppers who add items to their cart but do not complete the purchase, which helps pinpoint checkout process issues

Score: 4/10 | **Feedback:** You listed several relevant KPIs, but the question asked which KPIs you tracked in the specific Shop Easy project and required showing how you computed at least one KPI in SQL (definition/formula + joins/aggregation/date window). Your answer provides general KPI definitions but no project-specific selection, no formulas beyond high-level descriptions, and no SQL computation approach.

Q6: In the Shop Easy marketing analysis, how did you clean and validate the marketing data before reporting results (e.g., duplicates, inconsistent campaign names, missing values)?

Answer: In the Shop Easy marketing analysis, data cleaning and validation were performed directly within SQL to ensure the final KPIs were based on a "single version of truth". Domo Domo +1 The following steps were used to prepare the data: Deduplication: Repeated records can inflate performance metrics like total spend or clicks. Duplicates were identified and removed using ROW_NUMBER() within a Common Table Expression (CTE) to partition by unique identifiers (e.g., transaction_id or customer_id) and keep only the most recent entry. Standardizing Inconsistent Names: Marketing campaigns often had varying naming conventions (e.g., "SUMMER_SALE", "Summer Sale", "summer-sale"). These were unified using string functions like UPPER(), TRIM(), and REPLACE() to ensure all variations mapped to a single category for accurate grouping. Handling Missing Values: Null values in critical fields like email or discount_percentage were addressed by either filtering them out or imputing defaults. For example, COALESCE(discount, 0) was used to ensure calculations didn't break when a discount was not applied. Data Validation Rules: Range checks were applied to numeric fields (e.g., ensuring spend was never negative) and date formats were standardized using STR_TO_DATE or CAST to allow for proper time-series analysis. Referential Integrity Checks: Transactions were validated against the customer master table using LEFT JOINS to identify and remove "orphaned" records that lacked a corresponding valid customer ID.

Score: 8/10 | **Feedback:** Solid, practical overview of cleaning/validation steps with concrete SQL techniques (ROW_NUMBER, standardization functions, COALESCE, range/date checks, referential integrity). To be stronger, specify exact de-duplication keys aligned to the analysis grain (e.g., customer_id+event_id/order_id), describe a definitive campaign name mapping approach (lookup table/canonical taxonomy), and include reconciliation validation (row counts and KPI totals tied back to source/extract) plus spot checks.

Q7: In your 'Global Superstore Report (Power BI)', how did you model the data (tables/relationships), and which visuals or drilldowns did you use to identify top and bottom performers?

Answer: i dont know

Score: 0/10 | **Feedback:** The response does not address the question or demonstrate any knowledge of the data model, relationships, or visuals used in the Power BI report. Provide a clear description of the schema (fact/dim tables), relationship setup (including date table), and the specific visuals/drilldowns used to find top/bottom performers.

Q8: Describe a time you found a discrepancy between your SQL results and what stakeholders expected. How did you investigate and resolve it?

Answer: In the Shop Easy project, a significant discrepancy occurred when stakeholders expected a much higher Conversion Rate for a specific "Mid-Season" campaign than my SQL results showed. They suspected my query was undercounting successful orders.

Score: 2/10 | **Feedback:** You identified a discrepancy scenario (conversion rate vs stakeholder expectation) and the stakeholder hypothesis (undercounting orders), but you did not describe how you investigated, what you found (root cause), how you resolved it, or how you prevented recurrence. Add a structured debugging narrative (reproduce, isolate joins/filters, validate definitions, compare intermediate aggregates), plus the final fix and follow-up safeguards.

Q9: In the 'Customer Behavior & Product Performance Analytics (SQL)' warehouse project, explain your segmentation and product classification approach. How did you ensure segments were stable and meaningful over time?

Answer: In the 'Customer Behavior & Product Performance Analytics' warehouse project, I implemented a dynamic segmentation approach using RFM Analysis (Recency, Frequency, Monetary) to categorize users based on their transaction history. By utilizing SQL NTILE functions, I assigned percentile ranks to customers, grouping them into tiers such as "Champions," "At-Risk," and "Recent Lapsers." Simultaneously, products were classified using an ABC-XYZ Matrix, which combined total revenue contribution (ABC) with demand volatility (XYZ). To ensure these segments remained stable and meaningful over time, I moved away from static snapshots and implemented rolling time windows (e.g., 90-day activity cycles) and SCD Type 2 (Slowly Changing Dimensions) within the data warehouse. This allowed me to track segment migration, capturing when a customer transitioned from "Active" to "Churned," while maintaining historical integrity for year-over-year performance comparisons.

Score: 8/10 | **Feedback:** Strong, relevant approach: clear RFM segmentation and ABC-XYZ product classification, plus good handling of time via rolling windows and SCD2 to track migration. To make it more convincing, specify the exact scoring/banding rules (cutoffs, definitions for tiers like Champions/At-Risk), how seasonality was accounted for (e.g., same-period-last-year baselines or seasonal indices), and how you validated stability/meaningfulness (segment size distributions over time, transition matrices, lift/retention by segment, and business stakeholder checks).

Q10: If your warehouse project had to scale to 10x data volume and near-real-time KPI updates, what changes would you make to the data model, SQL pipelines, and reporting layer?

Answer: I DONT KNOW

Score: 0/10 | **Feedback:** The response does not address the question or provide any relevant strategies for scaling a warehouse to higher volume and near-real-time KPI updates. Even a high-level outline (e.g., incremental/CDC, partitioning, materialized views, orchestration, and reporting refresh/aggregations) was expected.